

Detailed mathematical write-up of the Lean formalisation O-minimal structures, definable maps, images, and local properties

1 Purpose and high-level structure

This document explains the Lean development in mathematical language. The Lean file builds an axiomatic version of an o-minimal structure over a densely ordered set, defines definable sets and definable functions, proves that several set-theoretic constructions preserve definability, and then uses these constructions to show that certain loci of one-variable definable functions are definable.

The additional result added to the Lean file is the following standard theorem.

Theorem 1.1 (Image of a definable function). *Let M be an o-minimal structure on a densely ordered set R . Let*

$$A \subseteq R^m, \quad B \subseteq R^n,$$

and let

$$f : A \rightarrow B$$

be a definable function. Then

$$\text{Im}(f) = \{y \in R^n : \exists x \in A, y = f(x)\}$$

is a definable subset of R^n . Moreover,

$$\text{Im}(f) \subseteq B.$$

The proof of this theorem does not use the special one-dimensional o-minimality axiom. It only uses the closure of definable sets under coordinate reindexing and existential projection. This matches the Lean proof: the graph of f is reindexed from coordinates (x, y) to coordinates (y, x) and then projected onto the y -coordinates.

2 Tuples, coordinates, and basic set operations

2.1 Tuple spaces

For each natural number n , the Lean file writes

$$\text{Power } R \text{ } n$$

for the type of n -tuples of elements of R . Formally this is implemented as

$$\text{Fin}(n) \rightarrow R.$$

This representation is convenient in Lean because coordinates are indexed by the finite type $\text{Fin}(n)$.

Mathematically, we write

$$\text{Power } R \text{ } n = R^n.$$

If $x \in R^n$, then x_i denotes the i -th coordinate of x .

2.2 Concatenation and block projections

If

$$x \in R^n, \quad y \in R^m,$$

then their concatenation is written

$$x \frown y \in R^{n+m}.$$

In Lean this is

`Power.append x y.`

If $z \in R^{n+m}$, then the left and right blocks are denoted by

$$z_L \in R^n, \quad z_R \in R^m.$$

In Lean these are

`Power.left z,` `Power.right z.`

They satisfy the expected identities

$$(x \frown y)_L = x, \quad (x \frown y)_R = y.$$

These are the Lean simp lemmas

`Power.left_append,` `Power.right_append.`

2.3 Basic Boolean operations on sets

A set in Lean is represented as a predicate. Thus a subset $A \subseteq R^n$ is implemented as

`Set (Power R n).`

The file defines the following elementary set operations directly as predicates:

`setEmpty =  $\emptyset$ ,`
`setUnion A B =  $A \cup B$ ,`
`setInter A B =  $A \cap B$ ,`
`setCompl A =  $(R^n \setminus A)$ .`

For example,

$$(\text{setInter } A \ B)(x) \iff A(x) \wedge B(x).$$

3 The ordered base set

Definition 3.1 (Dense linear order without endpoints). *A dense linear order without endpoints consists of a type R together with a binary relation $<$ such that the following properties hold.*

- (i) *Irreflexivity:* $\neg(x < x)$ for every $x \in R$.
- (ii) *Transitivity:* if $x < y$ and $y < z$, then $x < z$.
- (iii) *Trichotomy:* for all $x, y \in R$, exactly one of $x < y$, $x = y$, or $y < x$ holds.
- (iv) *Density:* if $x < y$, then there exists $z \in R$ such that $x < z < y$.

- (v) *No left endpoint: for every $x \in R$, there exists $y \in R$ such that $y < x$.*
- (vi) *No right endpoint: for every $x \in R$, there exists $z \in R$ such that $x < z$.*

In Lean this is the structure

DenseLinearOrderNoEndpoints.

The Lean development does not use the full power of an imported order typeclass. Instead, the order relation and the required axioms are packaged manually in the structure above. This keeps the formalisation self-contained.

4 Intervals and the one-dimensional o-minimality axiom

4.1 Endpoints

To state intervals uniformly, the Lean file introduces an endpoint type

$$R \cup \{-\infty, +\infty\}.$$

In Lean this is the inductive type

`Endpoint R,`

with constructors

`Endpoint.negInf,      Endpoint.finite a,      Endpoint.posInf.`

The endpoint order is the obvious extension of the order on R :

$$-\infty < a < +\infty$$

for every finite $a \in R$.

4.2 Points and intervals

For $a \in R$, the singleton $\{a\} \subseteq R$ is represented as a subset of R^1 :

$$\text{pointSet } a = \{x \in R^1 : x_0 = a\}.$$

For endpoints $a, b \in R \cup \{-\infty, +\infty\}$, the open interval is

$$(a, b) = \{x \in R : a < x < b\},$$

with the usual interpretation when one endpoint is infinite. In Lean this is

`openInterval D a b.`

Definition 4.1 (Finite unions of points and intervals). *A subset $A \subseteq R$ is a finite union of points and intervals if it belongs to the smallest class containing:*

- (i) *the empty set;*
- (ii) *every singleton $\{a\}$;*
- (iii) *every open interval (a, b) , with possibly infinite endpoints;*
- (iv) *the union of two members of the class.*

In Lean this is the inductive predicate

FiniteUnionOfPointsAndIntervals D A.

5 O-minimal structures in the Lean file

Definition 5.1 (Axiomatic o-minimal structure). *Let $(R, <)$ be a dense linear order without endpoints. An o-minimal structure on R , as formalised in the Lean file, consists of a family*

$$\mathcal{S}_n \subseteq \mathcal{P}(R^n), \quad n = 0, 1, 2, \dots,$$

satisfying the following closure and definability axioms.

(i) *Empty sets are definable:*

$$\emptyset \in \mathcal{S}_n.$$

(ii) *Definable sets are closed under finite unions:*

$$A, B \in \mathcal{S}_n \implies A \cup B \in \mathcal{S}_n.$$

(iii) *Definable sets are closed under finite intersections:*

$$A, B \in \mathcal{S}_n \implies A \cap B \in \mathcal{S}_n.$$

(iv) *Definable sets are closed under complements:*

$$A \in \mathcal{S}_n \implies R^n \setminus A \in \mathcal{S}_n.$$

(v) *Diagonals are definable: for $i < j < n$,*

$$\{x \in R^n : x_i = x_j\} \in \mathcal{S}_n.$$

(vi) *Products are definable: if $A \in \mathcal{S}_n$ and $B \in \mathcal{S}_m$, then*

$$A \times B = \{z \in R^{n+m} : z_L \in A \text{ and } z_R \in B\} \in \mathcal{S}_{n+m}.$$

(vii) *One-coordinate projections are definable: if $A \in \mathcal{S}_{n+1}$, then deleting any one coordinate gives a set in $\mathcal{S}_n$.*

(viii) *Coordinate reindexing preserves definability: if $A \in \mathcal{S}_n$ and $\sigma : \text{Fin}(n) \rightarrow \text{Fin}(m)$, then*

$$\{x \in R^m : (x_{\sigma(0)}, \dots, x_{\sigma(n-1)}) \in A\} \in \mathcal{S}_m.$$

(ix) *Block existential projection preserves definability: if $A \in \mathcal{S}_{n+m}$, then*

$$\{x \in R^n : \exists y \in R^m, x \frown y \in A\} \in \mathcal{S}_n.$$

(x) *The order relation is definable:*

$$\{(x, y) \in R^2 : x < y\} \in \mathcal{S}_2.$$

(xi) *O-minimality in dimension one:*

$$A \in \mathcal{S}_1 \iff A \text{ is a finite union of points and intervals.}$$

In Lean this structure is called

`OMinimalStructure`.

Remark 5.2. In many textbook treatments, some of the closure properties above are derived from a smaller list of axioms. In the Lean file they are included explicitly. This is not a mathematical restriction; it is a practical choice that avoids lengthy low-level proofs about finite coordinates.

Definition 5.3 (Definable set). Let M be an o -minimal structure. A subset $A \subseteq R^n$ is called definable if

$$A \in \mathcal{S}_n.$$

In Lean this is abbreviated by

`Definable M A`.

6 Definable functions

6.1 Graphs

Let

$$A \subseteq R^m, \quad B \subseteq R^n.$$

A function $f : A \rightarrow B$ is represented in Lean as a function between subtype spaces:

$$f : \{x : R^m \mid A(x)\} \longrightarrow \{y : R^n \mid B(y)\}.$$

Thus an input is not merely a tuple $x \in R^m$, but a pair consisting of x and a proof that $x \in A$. Likewise, the output contains both the value $f(x)$ and a proof that this value belongs to B .

The graph of f is a subset of R^{m+n} :

$$\text{Graph}(f) = \{(x, y) \in R^m \times R^n : x \in A \text{ and } y = f(x)\}.$$

In terms of concatenated tuples, if $z \in R^{m+n}$, then

$$z \in \text{Graph}(f) \iff \exists h : A(z_L), z_R = f(z_L, h).$$

This is the Lean definition

`FunctionGraph`.

Definition 6.1 (Definable function). A function $f : A \rightarrow B$ is definable if

- (i) $A \in \mathcal{S}_m$;
- (ii) $B \in \mathcal{S}_n$;
- (iii) $\text{Graph}(f) \in \mathcal{S}_{m+n}$.

In Lean this bundled object is

`DefinableFunction M A B`.

It contains the fields

`domain_mem`, `codomain_mem`, `toFun`, `graph_mem`.

7 Images of definable functions

This is the new part added to the Lean file.

7.1 The image set

Let

$$f : A \rightarrow B$$

be represented as a Lean function

$$f : \{x : R^m \mid A(x)\} \longrightarrow \{y : R^n \mid B(y)\}.$$

The image of f is the subset of R^n defined by

$$\text{Im}(f) = \{y \in R^n : \exists x \in R^m, \exists h : A(x), y = f(x, h)\}.$$

In Lean this is

$$\text{FunctionImage } f.$$

Written in ordinary mathematical notation, this is simply

$$\text{Im}(f) = \{y \in R^n : \exists x \in A, y = f(x)\}.$$

Lemma 7.1 (The image is contained in the codomain). *For any function $f : A \rightarrow B$,*

$$\text{Im}(f) \subseteq B.$$

Proof. Let $y \in \text{Im}(f)$. By definition of the image, there exist $x \in R^m$ and a proof $h : A(x)$ such that

$$y = f(x, h).$$

Since the codomain of f is the subtype $\{y : R^n \mid B(y)\}$, the output $f(x, h)$ comes with a proof that its underlying tuple belongs to B . Therefore $y \in B$. $\square$

This is the Lean theorem

$$\text{functionImage_subset_codomain.}$$

7.2 Why a coordinate swap is needed

The graph $\text{Graph}(f)$ is stored in coordinates

$$(x, y) \in R^{m+n}.$$

However, the image is obtained by keeping the y -coordinates and existentially quantifying over the x -coordinates:

$$y \in \text{Im}(f) \iff \exists x \in R^m, (x, y) \in \text{Graph}(f).$$

The available block projection axiom in the Lean file has the form

$$C \in \mathcal{S}_{n+m} \implies \{y \in R^n : \exists x \in R^m, y \frown x \in C\} \in \mathcal{S}_n.$$

Thus it projects away the last block. Since $\text{Graph}(f)$ is in the order (x, y) , we first reindex it into the order (y, x) .

Define the block-swap map

$$s : R^{n+m} \rightarrow R^{m+n}$$

by

$$s(y, x) = (x, y).$$

In the Lean file this is implemented by two definitions:

$$\text{imageGraphIndex}, \quad \text{imageGraphArg}.$$

The theorem

$$\text{imageGraphArg_append}$$

formalises the identity

$$s(y \smallfrown x) = x \smallfrown y.$$

7.3 Definability of the image

Theorem 7.2 (Definability of images). *Let M be an o-minimal structure. Let*

$$A \subseteq R^m, \quad B \subseteq R^n,$$

and let

$$f : A \rightarrow B$$

be a definable function. Then

$$\text{Im}(f) \in \mathcal{S}_n.$$

Equivalently, the image of a definable function is definable.

Proof. Since f is definable, its graph is definable:

$$\text{Graph}(f) \in \mathcal{S}_{m+n}.$$

Define the swapped graph

$$\text{Graph}(f)^{\text{sw}} \subseteq R^{n+m}$$

by

$$(y, x) \in \text{Graph}(f)^{\text{sw}} \iff (x, y) \in \text{Graph}(f).$$

Equivalently,

$$\text{Graph}(f)^{\text{sw}} = s^{-1}(\text{Graph}(f)),$$

where $s(y, x) = (x, y)$ is the block-swap map.

Because coordinate reindexing preserves definability and $\text{Graph}(f)$ is definable, we have

$$\text{Graph}(f)^{\text{sw}} \in \mathcal{S}_{n+m}.$$

Now apply block existential projection to the last m coordinates. We obtain the definable set

$$P = \{y \in R^n : \exists x \in R^m, (y, x) \in \text{Graph}(f)^{\text{sw}}\} \in \mathcal{S}_n.$$

By the definition of the swapped graph,

$$\begin{aligned} y \in P &\iff \exists x \in R^m, (y, x) \in \text{Graph}(f)^{\text{sw}} \\ &\iff \exists x \in R^m, (x, y) \in \text{Graph}(f) \\ &\iff \exists x \in A, y = f(x) \\ &\iff y \in \text{Im}(f). \end{aligned}$$

Therefore $P = \text{Im}(f)$, and since P is definable, $\text{Im}(f)$ is definable. □

In Lean the graph-only version is

`functionImage_mem_of_graph_mem.`

The bundled version for a `DefinableFunction` is

`functionImage_mem.`

The named image of a bundled definable function is

`DefinableFunction.image,`

and the two final bundled theorems are

`DefinableFunction.image_mem,` `DefinableFunction.image_subset_codomain.`

Remark 7.3. *The assumptions that A and B are definable are included in the bundled notion of `DefinableFunction`. The image-definability proof itself uses only definability of the graph. The codomain inclusion uses only the fact that the output type is the subtype B .*

8 Composition of definable functions

8.1 Relational composition

Let

$$G \subseteq R^{a+b}, \quad H \subseteq R^{b+c}$$

be definable relations. Their relational composition is

$$H \circ G = \{(x, z) \in R^{a+c} : \exists y \in R^b, (x, y) \in G \text{ and } (y, z) \in H\}.$$

In Lean this relation is

`RelationComp G H.`

Lemma 8.1 (Relational composition preserves definability). *If*

$$G \in \mathcal{S}_{a+b} \quad \text{and} \quad H \in \mathcal{S}_{b+c},$$

then

$$H \circ G \in \mathcal{S}_{a+c}.$$

Proof. Work in the larger coordinate space

$$R^{(a+c)+b}.$$

We write a point of this space as

$$(x, z, y),$$

where

$$x \in R^a, \quad z \in R^c, \quad y \in R^b.$$

Using coordinate reindexing, define the two sets

$$G' = \{(x, z, y) : (x, y) \in G\},$$

and

$$H' = \{(x, z, y) : (y, z) \in H\}.$$

Since G and H are definable and reindexing preserves definability, both G' and H' are definable. Therefore

$$G' \cap H'$$

is definable.

Project away the final block $y \in R^b$. The resulting set is

$$\{(x, z) : \exists y, (x, y) \in G \text{ and } (y, z) \in H\},$$

which is exactly $H \circ G$. Hence $H \circ G$ is definable. $\square$

The Lean proof is

`relation_comp_mem.`

The auxiliary definitions

`compGIndex, compHIndex, compGArg, compHArg`

encode the coordinate reindexings needed to read (x, y) and (y, z) from the larger tuple (x, z, y) .

8.2 Composition of functions

Proposition 8.2 (Composition of definable functions). *Let*

$$f : A \rightarrow B, \quad g : B \rightarrow C$$

be definable functions. Then

$$g \circ f : A \rightarrow C$$

is definable.

Proof. The domain of $g \circ f$ is A , which is definable because f is a definable function. The codomain is C , which is definable because g is a definable function.

It remains to prove that the graph is definable. For $x \in A$ and $z \in C$,

$$(x, z) \in \text{Graph}(g \circ f)$$

if and only if there exists $y \in B$ such that

$$(x, y) \in \text{Graph}(f) \quad \text{and} \quad (y, z) \in \text{Graph}(g).$$

Thus

$$\text{Graph}(g \circ f) = \text{Graph}(g) \circ \text{Graph}(f).$$

The right hand side is definable by relational composition. Hence $g \circ f$ is definable. $\square$

The Lean equality relating relational composition of graphs to the graph of the composed function is

`relationComp_functionGraph_eq.`

The bundled composition theorem is

`DefinableFunction.comp.`

9 Reusable definability devices for local properties

The second half of the Lean file proves definability of several one-variable loci. The proofs all follow the same pattern:

- (i) express the desired property using only existential quantifiers, Boolean operations, the graph of the function, the order relation, and diagonals;
- (ii) build a higher-dimensional counterexample set;
- (iii) project away the witness variables;
- (iv) take complements to express universal quantifiers;
- (v) project again if necessary.

The file introduces several helper predicates.

9.1 Reindexing a relation into a larger coordinate space

If $A \subseteq R^n$ and

$$\sigma : \text{Fin}(n) \rightarrow \text{Fin}(m),$$

then

$$\text{HoldsAt } A \text{ sigma}$$

is the subset of R^m defined by

$$\{z \in R^m : (z_{\sigma(0)}, \dots, z_{\sigma(n-1)}) \in A\}.$$

The theorem

$$\text{holdsAt_mem}$$

says that if A is definable, then $\text{HoldsAt } A \text{ sigma}$ is definable.

9.2 Domain, graph, order, and equality at chosen coordinates

For a one-dimensional domain $I \subseteq R$, the predicate

$$\text{DomainOn } I \ i$$

means that the i -th coordinate belongs to I :

$$z_i \in I.$$

For a graph $G \subseteq R^2$, the predicate

$$\text{GraphOn } G \ i \ j$$

means

$$(z_i, z_j) \in G.$$

The predicate

$$\text{LtOn } D \ i \ j$$

means

$$z_i < z_j.$$

The predicate

$$\text{EqOn } i \ j$$

means

$$z_i = z_j.$$

It is definable when $i < j$ by the diagonal axiom, and its complement is used to express inequality.

9.3 Intervals and cores

The helper predicate

$$\text{ValueIntervalAt } D \ a \ w \ b$$

means

$$z_a < z_w < z_b.$$

The helper predicate

$$\text{NeighbourhoodAt } D \ c \ x \ d$$

means

$$z_c < z_x < z_d.$$

Finally,

$$\text{CoreAt } D \ G \ x \ a \ b \ v$$

means

$$(z_x, z_v) \in G \quad \text{and} \quad z_a < z_v < z_b.$$

Each of these is definable because it is built from reindexed copies of G and of the order relation, using finite intersections.

The whole space R^n is definable because it is the complement of the empty set. This is the Lean theorem

$$\text{setUniv_mem.}$$

10 Continuity points

Let

$$I \subseteq R$$

be definable, and let

$$f : I \rightarrow R$$

be a definable function. Write

$$G = \text{Graph}(f) \subseteq R^2.$$

10.1 The intended mathematical predicate

A point $x \in R$ is a continuity point of f on I if $x \in I$ and for every open interval (a, b) containing $f(x)$, there exists an open interval (c, d) containing x such that

$$y \in I \text{ and } c < y < d \implies a < f(y) < b.$$

In graph language this becomes

$$x \in I \quad \text{and} \quad \forall a, b, v [G(x, v) \wedge a < v < b \implies \exists c, d (c < x < d \wedge \forall y, w (I(y) \wedge c < y < d \wedge G(y, w) \implies a < w < b))].$$

The Lean predicate encoding this mathematical statement is

ContinuousAtOnGraph.

10.2 The definable construction

The Lean proof constructs the discontinuity locus as an existential projection of a definable bad set. The coordinates in arity 8 are ordered as

$$(x, a, b, v, c, d, y, w).$$

Define the counterexample set

$$C_8 = \{(x, a, b, v, c, d, y, w) : y \in I, c < y < d, G(y, w), \neg(a < w < b)\}.$$

This is the Lean definition

Counter8.

It is definable by the theorem

counter8_mem.

Projecting away (y, w) gives a definable subset of R^6 :

$$C_6(x, a, b, v, c, d) \iff \exists y, w, (x, a, b, v, c, d, y, w) \in C_8.$$

This is

CounterExists6.

Its complement says that the neighbourhood (c, d) has no counterexample.

Next define the good six-dimensional set

$$G_6(x, a, b, v, c, d) \iff G(x, v) \wedge a < v < b \wedge c < x < d \wedge \neg C_6(x, a, b, v, c, d).$$

This is

Good6.

Projecting away (c, d) gives

$$E_4(x, a, b, v) \iff \exists c, d, G_6(x, a, b, v, c, d).$$

This is

GoodExists4.

Now define

$$B_4(x, a, b, v) \iff G(x, v) \wedge a < v < b \wedge \neg E_4(x, a, b, v).$$

This says that (a, b) is a bad neighbourhood of the value $v = f(x)$. It is represented by

Bad4.

Projecting away (a, b, v) gives the bad point set

$$B_1(x) \iff \exists a, b, v, B_4(x, a, b, v).$$

This is

BadPoint1.

The constructed continuity locus is

$$\text{Cont}(f) = I \cap (R \setminus B_1).$$

In Lean this is

ContinuousPoints.

Theorem 10.1 (Definability of the constructed continuity locus). *If $f : I \rightarrow R$ is definable, then **ContinuousPoints** is definable.*

Proof. The set C_8 is built from I , G , the order relation, finite intersections, and a complement. Hence C_8 is definable. By block projection, C_6 is definable. By complement and finite intersection, G_6 is definable. By projection, E_4 is definable. By complement and finite intersection, B_4 is definable. By projection, B_1 is definable. Finally,

$$I \cap (R \setminus B_1)$$

is definable by complement and intersection. □

The Lean theorem is

continuousPoints_mem.

Remark 10.2. *The Lean file defines both the intended pointwise predicate **ContinuousAtOnGraph** and the constructed set **ContinuousPoints**. The definability theorem uses the constructed set. The mathematical equivalence between the two is exactly the counterexample-unwinding described above.*

11 Locally constant points

11.1 The intended mathematical predicate

The function $f : I \rightarrow R$ is locally constant at x if $x \in I$ and there exists an interval (c, d) containing x such that

$$y \in I \text{ and } c < y < d \implies f(y) = f(x).$$

In graph language this is

$$\begin{aligned} x \in I \quad \text{and} \quad \exists c, d, v \quad & [G(x, v) \wedge c < x < d \\ & \wedge \forall y, w \quad (I(y) \wedge c < y < d \wedge G(y, w) \Rightarrow w = v)]. \end{aligned}$$

The corresponding Lean predicate is

LocallyConstantAtOnGraph.

11.2 The definable construction

Use coordinates

$$(x, c, d, v, y, w) \in R^6.$$

Define the bad set

$$C_6^{\text{lc}} = \{(x, c, d, v, y, w) : y \in I, c < y < d, G(y, w), w \neq v\}.$$

This is

$$\text{LocalConstCounter6.}$$

It is definable because $w \neq v$ is the complement of the diagonal $w = v$.

Projecting away (y, w) gives

$$C_4^{\text{lc}}(x, c, d, v),$$

which says that the proposed neighbourhood and value admit a counterexample. This is

$$\text{LocalConstCounterExists4.}$$

Its complement says that no such counterexample exists.

Define

$$G_4^{\text{lc}}(x, c, d, v) \iff G(x, v) \wedge c < x < d \wedge \neg C_4^{\text{lc}}(x, c, d, v).$$

This is

$$\text{LocalConstGood4.}$$

Projecting away (c, d, v) gives

$$G_1^{\text{lc}}(x) \iff \exists c, d, v, G_4^{\text{lc}}(x, c, d, v).$$

The final locally constant locus is

$$\text{LC}(f) = I \cap G_1^{\text{lc}}.$$

This is

$$\text{LocallyConstantPoints.}$$

Theorem 11.1 (Definability of the locally constant locus). *If $f : I \rightarrow R$ is definable, then the constructed locally constant locus is definable.*

Proof. The set C_6^{lc} is definable by reindexing I , G , and the order relation, and by using the complement of a diagonal to express inequality. Projection gives C_4^{lc} . Complement and intersection give G_4^{lc} . Projection gives G_1^{lc} . Intersecting with I gives $\text{LC}(f)$, which is therefore definable. $\square$

The Lean theorem is

$$\text{locallyConstantPoints_mem.}$$

12 Locally monotone points

12.1 Convention

The Lean file uses the non-strict convention:

$$\text{locally increasing} = \text{locally nondecreasing},$$

and

$$\text{locally decreasing} = \text{locally nonincreasing}.$$

Since the primitive order symbol is strict, the weak inequality $u \leq v$ is expressed as

$$\neg(v < u).$$

12.2 The intended mathematical predicates

The function $f : I \rightarrow R$ is locally increasing at x if $x \in I$ and there exist c, d such that $c < x < d$ and for all $y_0, y_1 \in I$,

$$c < y_0 < d, \quad c < y_1 < d, \quad y_0 < y_1 \implies f(y_0) \leq f(y_1).$$

Equivalently, in graph language, there should be no witnesses y_0, y_1, w_0, w_1 such that

$$G(y_0, w_0), \quad G(y_1, w_1), \quad y_0 < y_1, \quad w_1 < w_0.$$

Similarly, f is locally decreasing at x if there is no such local pair with

$$w_0 < w_1.$$

The corresponding Lean predicates are

$$\text{LocallyIncreasingAtOnGraph}, \quad \text{LocallyDecreasingAtOnGraph}.$$

12.3 The common base counterexample set

Use coordinates

$$(x, c, d, y_0, y_1, w_0, w_1) \in R^7.$$

The common base set is

$$C_7^{\text{base}} = \{(x, c, d, y_0, y_1, w_0, w_1) : y_0 \in I, y_1 \in I, c < y_0 < d, c < y_1 < d, \\ G(y_0, w_0), G(y_1, w_1), y_0 < y_1\}.$$

This is

$$\text{MonotoneBaseCounter7}.$$

It is definable by finite intersections and reindexing.

The increasing-bad set is

$$C_7^{\text{inc}} = C_7^{\text{base}} \cap \{w_1 < w_0\}.$$

This is

$$\text{MonoIncCounter7}.$$

The decreasing-bad set is

$$C_7^{\text{dec}} = C_7^{\text{base}} \cap \{w_0 < w_1\}.$$

This is

$$\text{MonoDecCounter7}.$$

Both are definable.

12.4 Projection to good neighbourhoods

Projecting away

$$(y_0, y_1, w_0, w_1)$$

gives definable subsets of R^3 :

$$C_3^{\text{inc}}(x, c, d), \quad C_3^{\text{dec}}(x, c, d).$$

They say that the interval (c, d) contains a bad pair for increasingness or decreasingness, respectively. In Lean these are

$$\text{MonoIncCounterExists3}, \quad \text{MonoDecCounterExists3}.$$

Their complements say that the proposed neighbourhood has no bad pair. Hence define

$$\begin{aligned} G_3^{\text{inc}}(x, c, d) &\iff c < x < d \wedge \neg C_3^{\text{inc}}(x, c, d), \\ G_3^{\text{dec}}(x, c, d) &\iff c < x < d \wedge \neg C_3^{\text{dec}}(x, c, d). \end{aligned}$$

These are the Lean definitions

$$\text{MonoIncGood3}, \quad \text{MonoDecGood3}.$$

Projecting away (c, d) gives one-dimensional good-point sets

$$G_1^{\text{inc}}, \quad G_1^{\text{dec}}.$$

These are

$$\text{MonoIncGoodPoint1}, \quad \text{MonoDecGoodPoint1}.$$

The final loci are

$$\text{LInc}(f) = I \cap G_1^{\text{inc}}, \quad \text{LDec}(f) = I \cap G_1^{\text{dec}}.$$

In Lean these are

$$\text{LocallyIncreasingPoints}, \quad \text{LocallyDecreasingPoints}.$$

Theorem 12.1 (Definability of local monotonicity loci). *If $f : I \rightarrow R$ is definable, then the constructed locally increasing and locally decreasing loci are definable.*

Proof. The base counterexample set is definable because it is built from I , G , and $<$ using reindexing and finite intersections. Intersecting it with $w_1 < w_0$ gives the increasing-bad set, and intersecting it with $w_0 < w_1$ gives the decreasing-bad set. Both are definable.

Projecting away the variables y_0, y_1, w_0, w_1 gives the bad-neighbourhood sets in R^3 . Taking complements expresses that no bad pair exists. Intersecting with $c < x < d$ and then projecting away (c, d) gives the points admitting a good neighbourhood. Finally, intersecting with I gives the locally increasing and locally decreasing loci. Each step preserves definability. $\square$

The Lean theorems are

$$\text{locallyIncreasingPoints_mem}, \quad \text{locallyDecreasingPoints_mem}.$$

13 Small sanity checks

The file ends with two simple checks.

- (i) The empty set is definable in every arity:

`empty_definable.`

- (ii) The order relation is definable:

`order_definable.`

14 Dictionary between Lean names and mathematical objects

Lean name	Mathematical meaning
<code>Power R n</code>	The tuple space R^n .
<code>Power.append</code>	Tuple concatenation $x \frown y$.
<code>Power.left</code> , <code>Power.right</code>	Left and right coordinate blocks of a concatenated tuple.
<code>setEmpty</code> , <code>setUnion</code> , <code>setInter</code> , <code>setCompl</code>	Empty set, union, intersection, complement.
<code>DenseLinearOrderNoEndpoints</code>	Dense linear order without endpoints.
<code>Endpoint R</code>	Formal endpoints $-\infty$, finite points of R , and $+\infty$.
<code>FiniteUnionOfPointsAndIntervals</code>	Finite unions of points and open intervals in dimension one.
<code>OMinimalStructure</code>	The family $(\mathcal{S}_n)_n$ of definable sets with the closure axioms and the one-dimensional o-minimality axiom.
<code>Definable M A</code>	The assertion $A \in \mathcal{S}_n$.
<code>FunctionGraph</code>	The graph of a function between subtype domains.
<code>DefinableFunction</code>	A bundled definable function with definable domain, definable codomain, and definable graph.
<code>FunctionImage</code>	The image set $\text{Im}(f)$.
<code>imageGraphIndex</code> , <code>imageGraphArg</code>	The coordinate swap used to turn graph coordinates (x, y) into projection-friendly coordinates (y, x) .
<code>imageGraphArg_append</code>	The identity $s(y \frown x) = x \frown y$ for the block-swap map.
<code>functionImage_mem_of_graph_mem</code>	If the graph is definable, then the image is definable.
<code>functionImage_mem</code>	The image of a bundled definable function is definable.

Lean name	Mathematical meaning
<code>DefinableFunction.image</code>	The named image of a bundled definable function.
<code>DefinableFunction.image_mem</code>	The named image is definable.
<code>DefinableFunction.image_subset_codomain</code>	The image is contained in the codomain.
<code>RelationComp</code>	Relational composition $H \circ G$.
<code>relation_comp_mem</code>	Relational composition preserves definability.
<code>relationComp_functionGraph_eq</code>	The relational composition of graphs equals the graph of the composed function.
<code>DefinableFunction.comp</code>	Composition of definable functions.
<code>HoldsAt</code>	Pullback of a definable set by a coordinate map.
<code>DomainOn</code>	The condition that a chosen coordinate lies in the domain I .
<code>GraphOn</code>	The condition that two chosen coordinates form a point of a graph G .
<code>LtOn</code>	The order condition $z_i < z_j$.
<code>EqOn</code>	The equality condition $z_i = z_j$.
<code>ContinuousPoints</code>	The constructed continuity locus.
<code>continuousPoints_mem</code>	Definability of the constructed continuity locus.
<code>LocallyConstantPoints</code>	The constructed locally constant locus.
<code>locallyConstantPoints_mem</code>	Definability of the constructed locally constant locus.
<code>LocallyIncreasingPoints</code>	The constructed locally nondecreasing locus.
<code>LocallyDecreasingPoints</code>	The constructed locally nonincreasing locus.
<code>locallyIncreasingPoints_mem,</code> <code>locallyDecreasingPoints_mem</code>	Definability of the constructed local monotonicity loci.

15 Summary of the new image theorem in one line

The mathematical content of the new Lean addition is exactly this implication:

$$\text{Graph}(f) \in \mathcal{S}_{m+n} \implies \{y \in R^n : \exists x \in R^m, (x, y) \in \text{Graph}(f)\} \in \mathcal{S}_n.$$

Because

$$\{y \in R^n : \exists x \in R^m, (x, y) \in \text{Graph}(f)\} = \text{Im}(f),$$

this proves that the image of a definable function is definable.